File Handling

- Introduction
- File Opening modes
- Opening and closing of a file
- Reading from a file
- Writing onto a file
- Error handling during I/O operations
- Record I/O in files

Introduction to files

- Programs are stored in memory i.e. Random access memory

But this storage is not permanent

- For permanent storage, it is required to use files to store programs and data and read them from there
- C language supports a number of functions to perform basic file operations
- A file is a collection of bytes that is stored on secondary storage devices like disk. There are two kinds of files in a system. They are,
- Text files (ASCII)
- Binary files
- Text files contain ASCII codes of digits, alphabets and symbols.
- Binary file contains collection of bytes (0's and 1's)

Difference between text files and binary files

- There are two kinds of storing data in file- text format and binary format
- In text file data are stored as line of characters with each line terminated by a new line character (‘\n’)
- In binary format data is stored on the disk in the same way as it is represented in the computer memory . Unix system does not make any distinction between text file and binary file.

- Text files are in human readable form and they can be created and read using any text editor, while binary files are not in human readable form and they can be created and read only by specific program written for them

Difference between text files and binary files contd..

- The binary data stored in file can not be read using text editor. The hexadecimal value of 1679 is 0x068f , so in binary format it is represented by two bytes 0x06 and 0x8f
- In a text file 1679 is represented by the bytes 0x31 , 0x36 , 0x37 ,0x39 (ASCII values) . NOTE ASCII value for 1 is 31 and for 9 is 39.

- Both text file and binary file keep record of the length of the file , and identify the end of file when this is reached
- In text files , there is one more way to detect the end of file. The character with ASCII value 26 is considered to be end of file character in text files . All input functions stop reading characters when this character is encountered and return end of file signal to the program
- C does not insert this character in file , it can be entered through the keyboard by Ctrl + Z (Ctrl + D on some systems) . In binary files no such character represent end of file.

Difference between text files and binary files contd..

- In text files newline is stored as a combination of carriage return ‘\r’ (ASCII 13) and linefeed ‘\n’(ASCII 10) while in binary files newline is stored only as ‘\n’(ASCII 10).

- In binary format , the data is stored in the same way as it is represented in memory so no conversions have to take place while transferring of data between memory and file
- In text format some conversions have to take place while transferring of data between memory and file.

  - For example while writing to a text file newline (‘\n’) has to be converted to combination of(‘\r’) and (‘\n’) and while reading from text file this combination (‘\r’) and (‘\n’) is to be converted back (‘\n’).

Difference between text files and binary files contd..

- The input and output operations in the binary files take less time as compared to the text files because no conversion has to be taken place
- However the data written using binary file system is not very portable because size of data type and byte order may be different on different machines
- In text format , these problems do not raise so it is considered more portable

- If the amount of numeric data stored in a disk is large, text mode is inefficient as numbers are stored as strings and hence more memory is required

File opening modes

- There are three file opening modes
- r Opens the file for reading only
- w Opens the file for writing only
- a Opens the file for appending (or adding) data to it
- If the purpose is **reading**, and if the file exists, then it is opened with the current contents; otherwise an error occurs
- When the opening mode is **writing**, a file with the specified name is created, if the file does not exist; the contents are deleted when the file already exists
- When the purpose is **appending**, the file is opened with the current contents; a file with the specified name is created, if the file does not exist

File opening modes contd..

- r+ the existing file is opened from the beginning for both reading and writing
- w+ Same as **w** except both for reading and writing
- a+ Same as **a** except both for reading and writing

Basic File Operations

- There are 4 basic operations that can be performed on any file in C programming language. They are,
- Opening/Creating a file
- Closing a file
- Reading a file
- Writing in a file

Opening / creating a file

- FILE *fp;
- fp=fopen (“filename”, ”mode”);
- A file with name filename is opened and assigns an identifier to fp
- This pointer fp contains all the information about the file and is used as link between the system and the program

Opening/Creating a file

- fp – file pointer to the data type “FILE”.
- filename – the actual file name with full path of the file.
- mode – refers to the operation that will be performed on the file. Example: r, w, a, r+, w+ and a+.

- fclose() – To close a file
- Declaration: int fclose(FILE *fp);
- fclose() function closes the file that is being pointed by file pointer fp. In a C program, we close a file as below.
- fclose (fp);
- All files are closed automatically when the program terminates
- Closing a file explicitly is a good programming practice

EXAMPLE PROGRAM FOR FILE OPEN, FILE WRITE AND FILE CLOSE

```
/ * Open, write and close a file : */
# include <stdio.h>
# include <string.h>

int main( )
{
    FILE *fp ;
    char data[50];
    /* opening file*/
    printf( "Opening the file test.c in write mode" ) ;
    fp = fopen("test.c", "w") ;
    if ( fp == NULL )
    {
        printf( "Could not open file test.c" ) ;
        return 1;
    }
```

EXAMPLE PROGRAM FOR FILE OPEN, FILE WRITE AND FILE CLOSE contd..

```
// printf( "\n Enter some text from keyboard” \
//          “ to write in the file testfile\n" ) ;
 printf( "\n Enter some text from keyboard to write in the file testfile\n" ) ;
/* getting input from user*/
/* accepts a line and stores in data*/
   while ( strlen ( gets( data ) ) > 0 )
   {
      /* writing in the file*/
      fputs(data, fp) ;
      fputs("\n", fp) ;
   }
   // closing the file
   printf("Closing the file test.c") ;
   fclose(fp) ;
   return 0;
}
```

OUTPUT:

Opening the file test.c in write mode

Enter some text from keyboard to write in the file test.c

Hello, How are you?

Closing the file test.c

EXAMPLE PROGRAM FOR FILE OPEN, FILE READ AND FILE CLOSE

```
/* Open, Read and close a file: reading string by string */

# include <stdio.h>
int main( )
{
        FILE *fp ;
        char data[50] ;
        printf( "Opening the file test.c in read mode\n" ) ;
        fp = fopen( "test.c", "r" ) ;
        if ( fp == NULL )
        {
                printf( "Could not open file test.c" ) ;
                return 1;
        }
        printf( "Reading the file test.c\n" ) ;
        while( fgets ( data, 50, fp ) != NULL )
        printf( "%s" , data ) ;
        printf("Closing the file test.c") ;
        fclose(fp) ;
        return 0;
}
```

EXAMPLE PROGRAM FOR FILE OPEN, FILE READ AND FILE CLOSE contd..

- OUTPUT:
- Opening the file test.c in read mode
- Reading the file test.c
- Hello, How are you?
- Closing the file test.c

File Input/Output Functions

Function name	Description
• fopen ()	function creates a new file or opens an existing file.
• fclose ()	function closes an opened file.
• getw ()	function reads an integer from file.
• putw ()	function writes an integer to file.
• fgetc ()	function reads a character from file.
• fputc ()	function writes a character to file.
• gets ()	function reads line from keyboard.
• puts ()	function writes line to o/p (output) screen.
• fgets ()	function reads string from a file, one line at a time.
• fputs ()	function writes string to a file.

File Input/Output Functions contd..

Function Description

- feof () finds end of file.
- fgetchar () reads a character from specified stream or file
- fprintf () writes formatted data to a file.
- fscanf () reads formatted data (set of data values) from a file.
- fputchar () writes a character onto the output stream or file.
- fseek() moves file pointer position to given location.
- SEEK_SET moves file pointer position to the beginning of the file.
- fread( ) reading data from file opened by fopen function
- fwrite( ) writing data to the file opened by fopen function

File Input/Output Functions contd..

- SEEK_CUR moves file pointer position to given location.
- SEEK_END moves file pointer position to the end of file.
- ftell () function gives current position of file pointer.
- rewind () moves file pointer position to the beginning of the file.
- getc () reads character from file.
- getch () reads character from keyboard which may be different from stdin ; in conio.h header file
- getche () reads character from keyboard and echoes to screen.
- getchar () reads character from keyboard
- putc () writes a character to file.
- putchar () writes a character to screen

Functions for Random Access to Files

- ftell
- rewind
- fseek
- n = ftell (fp);
- n is the number of bytes already read by file pointer fp

- rewind(fp); takes a file pointer and resets the position to the start of the file
- The function helps in reading a file more than once, without having to close and open the file

- fseek(fileptr, offset, position);

- fileptr is a pointer to the file

- offset specifies the number of positions (bytes) to be moved from the location specified by position

Read/Write structure to a file using C

- fwrite() and fread() are used to write to a file and read from a file in C.
- **fwrite() syntax**
- fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream)
- ptr - A pointer to array of elements to be written
- size - Size in bytes of each element to be written
- nmemb - Number of elements, each one with a size of bytes
- stream – A pointer to a FILE object that specifies an output stream

fread() syntax

- fread(void *ptr, size_t size, size_t nmemb, FILE *stream)
- ptr - A pointer to a block of memory with a minimum size of size*nmemb bytes.
- size - Size in bytes of each element to be read.
- nmemb - Number of elements, each one with a size of bytes.
- stream - A pointer to a FILE object that specifies an input stream.

Algorithm to read/write structure in C

- Begin
- Create a structure Student to declare variables.
- Open file to write.
- Check if any error occurs in file opening.
- Initialize the variables with data.
- If file open successfully, write struct using write method.
- Close the file for writing.
- Open the file to read.
- Check if any error occurs in file opening.
- If file opens successfully, read the file using read method.
- Close the file for reading.
- Check if any error occurs.
- Print the data.
- End

Example program to read/write structure in C

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
struct Student {
   int roll_no;
   char name[20];
};
int main () {
   FILE *fp;
   fp= fopen ("c1.txt", "w");
   if (fp == NULL) {
      fprintf(stderr, "\nError to open the file\n");
      exit (1);
   }
```

Example program to read/write structure in C contd..

- struct Student inp1 = {1, "Ram"};
- struct Student inp2 = {2, "Shyam"};
- fwrite (&inp1, sizeof(struct Student), 1, fp);
- fwrite (&inp2, sizeof(struct Student), 1, fp);
- if(fwrite != 0)
- printf("Contents to file written successfully !\n");
- else
- printf("Error writing file !\n");
- fclose (fp);

Example program to read/write structure in C contd..

```
FILE *inf;
    struct Student inp;
    inf = fopen ("c1.txt", "r");
    if (inf == NULL) {
        fprintf(stderr, "\nError to open the file\n");
        exit (1);
    }
    while(fread(&inp, sizeof(struct Student), 1, inf))
        printf ("roll_no = %d name = %s\n", inp.roll_no, inp.name);
    fclose (inf);
}
```

Example program to read/write structure in C contd..

- Ouput
- Contents to file written successfully !
- roll_no = 1 name = Ram
- roll_no = 2 name = Shyam

EOF, feof() and ferror() functions

- EOF is a negative integer constant that indicates the end of a stream; often it is -1
-
- EOF is returned when end of file is reached. Only comparing the value returned by getc() with EOF is not sufficient to check for actual end of file as sometimes EOF is returned due to some other reason also
- To solve this problem, C provides feof() which returns non-zero value only if end of file has reached, otherwise it returns 0.
- feof( ) can be used to test for end of file condition
- FILE * fp
- feof(fp) where fp is a file pointer, returns a non zero integer value if all the data from file is read
- Returns zero if data is not completely read or end of file is not reached
- if feof (fp))

  printf (“End of data\n”);

  would display the message “End of data” on reaching the end of file

Note: In C, anything non zero is true

ferror ( ) function

- ferror( ) function gives the status of the file
- ferror(fp) returns a non zero integer if an error has been detected up to that point
- ferror(fp) returns a zero if no error is detected

if ferror(fp!=0)

printf(“An error has occurred\n”);

would print the message “An error has occurred” if processing is not successful

Error Handling in Files during I/O operations

- When trying to read a file beyond indicator.
- When trying to read a file that does not exist.
- When trying to use a file that has not been opened.
- When trying to use a file in an appropriate mode i.e., writing data to a file that has been opened for reading.

Example to demonstrate error handling

```
#include <stdio.h>
int main () {
   FILE *fp;
   int c;
   fp = fopen("testfile","r");
   if(fp == NULL) {
      perror("Error in opening file");   //perror is Posix error function
      return(-1); }
   while(1) {
      c = fgetc(fp);
      if( feof(fp) ) {
         break ;
      }    printf("%c", c);
   }
   fclose(fp);  return(0); }
```

Example program to illustrate error handling

```
#include <stdio.h>
 main ( )
{
   char * filename;
    FILE * fp1, *fp2;
    int i, number;
     fp1 = fopen (“TEST”, “w”);
      for(i=10; i<=100; i+10)
      putw(i,fp1); /* write integer i into the file “TEST”*/
      fclose(fp1);
```

Example program to illustrate error handling contd..

```
printf ("\n Please type input filename");
 open_file:   /* open_file is a lable*/
   scanf("%s", filename);
 if (fp2= fopen(filename, "r") == NULL)
 {
 printf("Cannot open the file.\n");
 printf("Type file name again.\n\n");
 goto open_file; /* go to the above lable to accept filename again*/
}
 else
   for (i=10; I <=100; i+10)
    {  number=getw(fp2);  /* read integer from file and store in number*/
       if (feof(fp2) {         /* if end of file is true*/
                  printf("\n Ran out of data.\n");
                   break; }  /* end if*/
      else
          printf ("%d\n", number);    }    /* end for*/
  fclose(fp2);
}   /* end of main*/
```

Output of program to illustrate error handling

- Please type input filename
- TETS suppose you gave TETS instead of TEST
- Cannot open the file.
- Type filename again.

- TEST This time you gave correct filename
- 10
- 20
- 30
- 40
- 50
- 60
- 70
- 80
- 90
- 100

- Ran out of data.